

Exercise 5: Update an Editable Grid

Interfaces 102: Building Dynamic Interfaces

Introduction	2
How Can I Practice?	2
Appian Version	2
Naming Conventions	2
Save Often	2
Additional Resources	2
Examine an Editable Grid	3
Add a New Column to the Editable Grid	6
Configure the Cost Column	8
Example Interface Expression	10

Notice of Rights

This document was created by Appian Corporation, 7950 Jones Branch Dr, Tysons, Virginia 22102. Copyright 2026 by Appian Corporation. All rights reserved. Information in this document is subject to change. No part of this document may be reproduced or transmitted in any form by any means, without prior written permission of Appian Corporation. For more information on obtaining permission for reprints or excerpts, contact Appian Training at academyonline@appian.com.

Introduction

How Can I Practice?

You should practice in Appian Community Edition. Keep in mind that this is a community environment not suitable for production workloads or sensitive information.

Appian Version

Appian Community Edition is on the latest Appian version. If you are following the exercises from a previous Appian version, go to academy.appian.com to download the latest version.

Naming Conventions

When you register for Appian Community Edition, you gain access to a workspace, which is your personal area within the community environment. When your workspace has been assigned, you will receive a confirmation email with a workspace ID. Pre-loaded apps in your workspace include the workspace ID.

Use this workspace ID when creating new applications and objects. Otherwise, you could run into naming conflicts with objects created by other users. In the example below, the application prefix for the Acme Auto Solution app is W0000AS. So, the workspace ID is W0000, which would be included in any new app's prefix.

Save Often

Appian does not automatically save updates, so save your objects frequently.

Additional Resources

Appian provides a number of training resources for Appian developers. The following resources are particularly popular with our learners:

- [Academy Online](#) - Appian's online courses provide useful survey courses, step-by-step tutorials, and practice exercises. Explore these resources at your own pace. Survey courses will help you develop a better grasp of the topics you need to learn. Video and print tutorials will help you with getting hands-on experience with Appian.
- [Community Discussions for New Users](#) - Check out the **New to Appian** thread in Community. Join our community of experts to ask questions and find answers from past discussions.
- [Appian Documentation](#) - Appian's product documentation will provide you with an overview of key Appian features, newest release information, additional tutorials, and helpful patterns and recipes to implement in your app.

Examine an Editable Grid

In the last exercise, you added links to the cards that allowed users to select a maintenance request. You also added dynamic styling to the cards to indicate the user selection. So far, all the updates to this interface have happened on the first page. In this exercise, you'll turn your attention to the second step which includes an editable grid with which users can add parts to the estimate. First, you'll examine the editable grid as it is, before making updates.

1. Switch to the second step of the wizard by clicking **NEXT** in the live preview.



2008 Dodge Sprinter

The transmission fluid needs to be changed as it is beginning to become dirty and could cause damage to the transmission if not addressed.

[CANCEL](#)[NEXT](#)

2. Try out the editable grid by clicking **Add Row**, then entering some sample values in each field.

Part

Enter parts for the estimate

Name	Unit Cost	Quantity
Rear Passenger Door	655.74	1

[+ Add Row](#)

3. You will see `ri!estimate` updated with your values in the **Rule Inputs** pane.

▼ Rule Inputs +	
Name	Value
estimate	[W5656ADX Estimate p...
maintenancelid	74
part (1 item)	[[W5656ADX Part name...
[1]	[W5656ADX Part name...
name	Rear Passenger Door
unitCost	655.74
quantity	1
cancel	null

4. Click the editable grid in the preview to navigate to the **`a!gridLayout()`** in the interface expression.

```
a!wizardStep(  
  label: "Part",  
  contents: {  
    a!cardLayout(  
      contents: {  
        a!gridLayout(  
          label: "Part",  
          labelPosition: "COLLAPSED",  
          instructions: "Enter parts for the estimate",  
          height: "AUTO",  
          borderStyle: "LIGHT",  
          shadeAlternateRows: false,  
          rowHeader: 1,  
          headerCells: {↔},  
          columnConfigs: {↔},  
          rows: a!forEach(↔),  
          addRowLink: a!dynamicLink(↔)  
        )  
      },  
      shape: "SEMI_ROUNDED",  
      padding: "MORE",  
      marginBelow: "STANDARD",  
      style: "NONE"  
    )  
  }  
)  
)
```

Note: Portions of the expression above have been collapsed for readability.

5. Look at the rows parameter. Notice that it uses an **`a!forEach()`** function with the **items** parameter set to the relationship **part** of the rule input **`ri!estimate`**. The expression argument is set to an **`a!gridRowLayout()`** function.

```

rows: a!forEach(
  items: ri!estimate[W5656ADX Estimate.part],
  expression: a!gridRowLayout(
    id: fv!index,
    contents: {↔}
  )
),

```

This means that this **a!forEach()** loop will create one new **a!gridRowLayout()** (used to generate a row in the grid) for each part related to the estimate.

- Now, examine the contents parameter of the **a!gridRowLayout()**. Here you will see four fields: one for each column in the grid.

```

rows: a!forEach(
  items: ri!estimate[W5656ADX Estimate.part],
  expression: a!gridRowLayout(
    id: fv!index,
    contents: {
      a!textField(↔),
      a!floatingPointField(↔),
      a!integerField(↔),
      a!imageField(↔)
    }
  )
),

```

Note: The fields' expressions have been collapsed in this image for better readability.

- Now, back up to the **a!gridLayout()** function, examine the **headerCells**, and **columnConfigs** parameters. Notice that each of these are lists with four items: one for each column in the grid.

```

a!gridLayout(
  label: "Part",
  labelPosition: "COLLAPSED",
  instructions: "Enter parts for the estimate",
  height: "AUTO",
  borderStyle: "LIGHT",
  shadeAlternateRows: false,
  rowHeader: 1
  headerCells: {
    a!gridLayoutHeaderCell(label: "Name"),
    a!gridLayoutHeaderCell(label: "Unit Cost"),
    a!gridLayoutHeaderCell(label: "Quantity"),
    a!gridLayoutHeaderCell()
  },
  columnConfigs: {
    a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
    a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
    a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
    a!gridLayoutColumnConfig(width: "ICON")
  },
  rows: a!forEach(
    items: ri!estimate[ W5656ADX Estimate.part ],
    expression: a!gridRowLayout(
      id: fv!index,
      contents: {
        a!textField(↔),
        a!floatingPointField(↔),
        a!integerField(↔),
        a!imageField(↔)
      }
    )
  ),
  addRowLink: a!dynamicLink(↔)
)

```

You will make changes in these three locations (**headerCells**, **columnConfigs**, and the contents of **a!gridRowLayout()**) to add a new column to the grid.

Add a New Column to the Editable Grid

Now, you will add a new column to the grid that displays the total cost of a given row. For example if you have a part that costs \$10, and you need 5 of that part, you would want the total cost of \$50 to display.

1. In the **headerCells** parameter of the **a!gridLayout()**, add another **a!gridLayoutHeaderCell()** function below the one for quantity. Give the column the label **"Cost"**.

```
headerCells: {
  a!gridLayoutHeaderCell(label: "Name"),
  a!gridLayoutHeaderCell(label: "Unit Cost"),
  a!gridLayoutHeaderCell(label: "Quantity"),
  a!gridLayoutHeaderCell(label: "Cost"),
  a!gridLayoutHeaderCell()
},
```

Note: This will cause an error in your interface. This error will persist until all three locations (**headerCells**, **columnConfigs**, and the contents of **a!gridRowLayout()**) have been updated.

2. In the **columnConfigs** parameter of the **a!gridLayout()**, add another **a!gridLayoutColumnConfig()** function above the last one. Don't specify any parameters in the function and delete the **width** parameter of the two **a!gridLayoutColumnConfig()** functions above the one you added.

```
columnConfigs: {
  a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
  a!gridLayoutColumnConfig(),
  a!gridLayoutColumnConfig(),
  a!gridLayoutColumnConfig(),
  a!gridLayoutColumnConfig(width: "ICON")
},
```

Deleting the specified widths will allow the respective columns to take up less space on screen.

3. In the **contents** parameter of the **a!gridRowLayout()**, add an **a!textField()** above the **a!imageField()**.

```
rows: a!forEach(
  items: ri!estimate[ W5656ADX Estimate.part ],
  expression: a!gridRowLayout(
    id: fv!index,
    contents: {
      a!textField(),
      a!floatingPointField(),
      a!integerField(),
      a!textField(),
      a!imageField()
    }
  )
),
```

Note: Adding the **a!textField()** should finally clear the error in your interface.

Your grid should now have four columns. Next, you will configure the final column to display the correct information.

Enter parts for the estimate

Name	Unit Cost	Quantity	Cost
Rear Passenger Door	655.74	1	

 Add Row

Configure the Cost Column

If you examine any of the fields in the `a!gridColumnLayout()` function, you'll see that they are using the function variable `fv!index` in the value and saveInto parameters. Remember that the rows are being created in an `a!forEach()` loop that is looping through a list of W#ADX Part records.

```
rows: a!forEach(  
  items: ri!estimate[ W5656ADX Estimate.part ],  
  expression: a!gridRowLayout(  
    id: fv!index,  
    contents: {  
      a!textField(),  
      a!floatingPointField(),  
      a!integerField(  
        value: fv!item[ W5656ADX Part.quantity ],  
        saveInto: fv!item[ W5656ADX Part.quantity ]  
      ),  
      a!textField(),  
      a!imageField()  
    }  
  )  
)
```

Next, you'll do something similar to populate the **`a!textField()`** you added to the grid.

1. In the **a!textField()**, set the **readOnly** parameter to **true**.

```
expression: a!gridRowLayout(  
  id: fv!index,  
  contents: {  
    a!textField(↔),  
    a!floatingPointField(↔),  
    a!integerField(↔),  
    a!textField(  
      readOnly: true,  
    ),  
    a!imageField(↔)  
  }  
)
```

Since this field is calculated based on the values of other fields, there is no need for the user to enter a value manually. For the same reason, you will not need to use the **saveInto** parameter.

2. Set the **value** parameter to use the **currency()** function. This function formats a supplied decimal as a string with the appropriate currency symbol and other formatting. In the currency function, set the following parameters.
 - **isoCode:** "USD"
 - **value:** **fv!item[W#ADX Part.quantity] * fv!item[W#ADX Part.unitCost]**

```
a!textField(  
  readOnly: true,  
  value: a!currency(  
    isoCode: "USD",  
    value: fv!item[W5656ADX Part.quantity] * fv!item[W5656ADX Part.unitCost]  
  )  
)
```

If you have already added a part in your interface, you should see the correct calculated value for the total cost for the row. However, if you try to add a new row, you'll see the following error.

! Could not display interface. Please check definition and inputs.

Interface Definition: Expression evaluation error at function a!wizardLayout [line 3]: Error in a!forEach() expression during iteration 2: Expression evaluation error at function a!currency parameter 2 [line 128]: Cannot mul incompatible operands of type Null and type Null.

3. This error arises when trying to multiply two values of type Null. To resolve this error, wrap both values in the function **todecimal()** as below.

```
a!textField(  
  readOnly: true,  
  value: a!currency(  
    isoCode: "USD",  
    value: todecimal(fv!item[W5656ADX Part.fields.quantity]) * todecimal(fv!item[W5656ADX Part.unitCost])  
  )  
)
```

4. Add a new row and verify that the Cost column is working correctly.
5. Save your changes.

Example Interface Expression

A full example expression of what your interface might look like at this stage is available below. Keep in mind that you will not be able to copy and paste this into your interface as record type field references are unique to each workspace.

```
a!localVariables(  
  local!availableMaintenanceRequests:  
    rule!W#ADX_getRequestedMaintenance(),  
  a!wizardLayout(  
    titleBar: a!headerTemplateFull(  
      title: "Create Estimate",  
      secondaryText: "Enter details for the estimate"  
    ),  
    isTitleBarFixed: true,  
    showButtonDivider: true,  
    backgroundColor: "#F6F7F9",  
    steps: {  
      a!wizardStep(  
        label: "Details",  
        contents: {  
          a!cardLayout(  
            contents: {  
              a!cardGroupLayout(  
                cards: a!forEach(  
                  items: local!availableMaintenanceRequests,  
                  expression: a!cardLayout(  
                    padding: "NONE",  
                    link: a!dynamicLink(  
                      value: fv!item[W#AA Maintenance.id],  
                      saveInto: ri!estimate[W#ADX Estimate.maintenanceId]  
                    ),  
                    style: a!match(  
                      value: ri!estimate[W#ADX Estimate.maintenanceId],  
                      equals: fv!item[W#AA Maintenance.id],  
                      then: "ACCENT",  
                      default: "NONE"  
                    ),  
                    contents: {  
                      a!columnsLayout(  
                        columns: {  
                          a!columnLayout(  
                            width: "NARROW",  
                            contents: a!imageField(  
                              labelPosition: "COLLAPSED",  
                              images: a!documentImage(  
                                document: if(  
                                  condition: a!isNullOrEmpty(  
                                    fv![W#AA  
                                      Maintenance.vehicle.image]  
                                  ),  
                                  valueIfTrue:  
                                    a!EXAMPLE_DOCUMENT_IMAGE(),  
                                )  
                              )  
                            )  
                          )  
                        )  
                      )  
                    )  
                  )  
                )  
              )  
            )  
          )  
        )  
      )  
    }  
  )  
)
```

```

        valueIfFalse:
            fv![W#AA
                Maintenance.vehicle.image]
        )
    )
)
),
a!columnLayout(
    contents: {
        a!headingField(
            text: {
                fv!item[W#AA Maintenance.vehicle.year]
                & " " &
                fv!item[W#AA Maintenance.vehicle.make]
                & " " &
                fv!item[W#AA Maintenance.vehicle.model]
            },
            size: "MEDIUM"
        ),
        a!richTextDisplayField(
            labelPosition: "COLLAPSED",
            value: fv!item[W#AA Maintenance.issue]
        )
    }
)
)
)
)
)
)
)
),
shape: "SEMI_ROUNDED",
padding: "MORE",
marginBelow: "STANDARD",
style: "NONE"
)
}
),
a!wizardStep(
    label: "Part",
    contents: {
        a!cardLayout(
            contents: {
                a!gridLayout(
                    label: "Part",
                    labelPosition: "COLLAPSED",
                    instructions: "Enter parts for the estimate",
                    height: "AUTO",
                    borderStyle: "LIGHT",
                    shadeAlternateRows: false,
                    rowHeader: 1,
                    headerCells: {

```

```

a!gridLayoutHeaderCell(label: "Name"),
a!gridLayoutHeaderCell(label: "Unit Cost"),
a!gridLayoutHeaderCell(label: "Quantity"),
a!gridLayoutHeaderCell(label: "Cost"),
a!gridLayoutHeaderCell()
},
columnConfigs: {
a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
a!gridLayoutColumnConfig(),
a!gridLayoutColumnConfig(),
a!gridLayoutColumnConfig(),
a!gridLayoutColumnConfig(width: "ICON")
},
rows: a!forEach(
items: ri!estimate[W#ADX Estimate.part],
expression: a!gridRowLayout(
id: fv!index,
contents: {
a!textField(
value: fv!item[W#ADX Part.name],
saveInto: fv!item[W#ADX Part.name]
),
a!floatingPointField(
value: fv!item[W#ADX Part.unitCost],
saveInto: fv!item[W#ADX Part.unitCost],
),
a!integerField(
value: fv!item[W#ADX Part.quantity],
saveInto: fv!item[W#ADX Part.quantity]
),
a!textField(
readOnly: true,
value: a!currency(
isoCode: "USD",
value: todecimal(fv!item[W#ADX Part.quantity]) *
todecimal(fv!item[W#ADX Part.unitCost])
)
),
a!imageField(
size: "ICON",
images: a!documentImage(
document: a!iconIndicator(icon: "REMOVE"),
altText: "Remove Row" & char(32) & fv!index + 1,
link: a!dynamicLink(
value: fv!index,
saveInto: {
a!save(
ri!estimate[W#ADX Estimate.part],
remove(
ri!estimate[W#ADX Estimate.part],
save!value
)
)
)
)
)
)
)

```

```

        }
    )
}
)
),
addRowLink: a!dynamicLink(
    label: "Add Row",
    value: W#ADX Part(),
    saveInto: a!save(
        ri!estimate[W#ADX Estimate.part],
        append(
            ri!estimate[W#ADX Estimate.part],
            save!value
        )
    )
)
),
),
),
),
),
),
),
shape: "SEMI_ROUNDED",
padding: "MORE",
marginBelow: "STANDARD",
style: "NONE"
)
}
)
},
primaryButtons: a!buttonWidget(
    label: "Create",
    submit: true,
    validate: true,
    style: "SOLID",
    showWhen: fv!isLastStep
),
secondaryButtons: a!buttonWidget(
    label: "Cancel",
    submit: true,
    validate: false,
    saveInto: ri!cancel,
    style: "LINK",
    value: true
),
style: "DOT_VERTICAL",
showTitleBarDivider: false,
contentsWidth: "FULL"
)
)
)

```